

Proof-Driven Enterprise Transformation Program

A controlled, evidence-bound execution system for evolving Stoquify into an enterprise-grade POS, inventory, accounting, purchasing, HRIS/payroll, compliance, and operational assurance platform.

Program decision	GO_WITH_GATES for staged transformation
Production decision	NO_GO until Phase 8 passes
Delivery model	Sequential implementation; parallel read-only analysis
Human control	Explicit approval after every phase
Evidence standard	Markdown, PDF, and machine-readable gate evidence
Date	24 July 2026

Core principle: every material workflow must pass through authorization, validation, an idempotent transaction, authoritative domain truth, event/outbox evidence, provenance, assurance, exception ownership, trusted read models, and evidence invalidation when source truth changes.

1. Program Summary

Transform the existing enterprise assessment into a reusable execution system consisting of one master orchestrator prompt, four delivery stages, nine dependency-ordered phases, one authoritative skill suite per phase, specialist agents for discovery/implementation/verification, and explicit human gates.

- One master program-orchestrator prompt.
- Four delivery stages containing nine dependency-ordered phases.
- One authoritative Stoquify skill suite per phase.
- Specialist agents for discovery, implementation, and independent verification.
- A sequential implementation spine with parallel read-only analysis only.
- Durable Markdown, PDF, and JSON evidence for every phase gate.

2. Operating Model

1. Set /permissions to Read Only and reasoning to High/XHigh.
2. Read the program register, applicable AGENTS.md, assessment, current Git state, and architecture graphs.
3. Run up to three parallel read-only specialist agents for architecture, security, and domain evidence.
4. Produce one bounded implementation slice with success criteria, affected boundaries, rollback, and verification.
5. Record a human GO, GO_WITH_GATES, or NO_GO.
6. Set /permissions to Workspace/Auto for one implementation owner only.
7. Implement surgically; no parallel overlapping writers.
8. Run focused tests, domain gates, cross-domain assurance, /diff, and /review.
9. Have an independent read-only verifier assess the evidence.
10. Save the handoff packet, update the program register, run /compact, and stop for approval.

Permissions and reasoning defaults

- Discovery: Read Only + XHigh.
- Implementation: Workspace + Medium/High.
- Security, accounting, migration, and concurrency decisions: High/XHigh.
- Verification: Read Only + High.
- Network, production, deployment, external writes, shared-database migrations, or destructive operations: explicit approval only.

3. Stage A — Establish Authoritative Control

0. Evidence Baseline	Reconcile dirty worktree, stale reports, current gates, graph freshness, and commit-bound evidence.	001-aqstoqflow-program-orchestrator; aqstoqflow-baseline-refresh-ratchet; aqstoqflow-release-verification-foundation	Program Orchestrator; Git Workflow Master; Reality Checker	Deliberately partitioned worktree; current SHA; refreshed evidence manifest; reproducible gates; no lost user changes
1. Module Entitlement Control Plane	Create durable tenant/module entitlement truth and pilot read-only Analytics enforcement.	module-control-plane-orchestrator; entitlement-schema; tenant-entitlement-provisioning; access-guard-contract; enforcement-pilot; release-gates-and-rollback	Software Architect; Security Architect; Product Manager; Backend Architect	Persistent lifecycle; deterministic backfill; no RBAC bypass; Analytics pilot certified; rollback proven
2. Domain Truth and Writer Registry	Make authoritative ownership, allowed writers, events, and gateways machine-verifiable.	002-aqstoqflow-control-plane; 004-aqstoqflow-business-event-gateway; service-boundary-burndown; module-surface-registry-ratchet	Backend Architect; Data Engineer; Code Reviewer	One owner per critical model; unauthorized writers fail ratchet; material commands produce correlated events/outboxes

4. Stage B — Close the Financial Proof Spine

3. Close Invalidation Mesh	Prove every close-impacting change invalidates certified evidence or is explicitly non-impacting.	close-invalidation-completion; 023 close-assurance-suite; 022 close-pack-certification	CFO; Financial Analyst; Compliance Auditor; Reality Checker	Machine-readable writer/invalidation registry; focused tests; stale-pack blocking; reversal and recertification proof
4. POS and Inventory Certification	Certify online/offline POS, exact-once replay, stock truth, valuation, corrections, and recovery.	offline-pos-sync-architect-builder; offline-pos-replay-finalizer; inventory-boundary-gate; inventory-valuation-kernel; stock-adjustment-writeoff-finalizer	Backend Architect; SRE; Security Architect; Evidence Collector	Device/replay chaos; deterministic reservation identity; projection rebuild; valuation tie-out; POS-to-close proof
5. Purchasing, AP, Cash and Reconciliation	Certify procure-to-pay, supplier-destination controls, settlement, suspense, and close integration.	purchasing-ap-controls; ap-gate-closer; ap-finalizer; payment-reconciliation-moat; reconciliation-proof-launcher; provider-health-recon-ops	CFO; Backend Architect; Application Security Engineer; Test Results Analyzer	Maker-checker and three-way-match proof; provider sandbox evidence; settlement tie-out; exception ownership; AP-to-close certification

5. Stage C — Complete Workforce and Statutory Truth

6. HRIS Ownership and Payroll Closure	Establish HRIS-owned people truth and recertify HRIS-to-payroll-to-accounting workflows.	aqstoqflow-hris-payroll-00-orchestrator followed by numbered status, truth-map, identity, org, contract, compensation, evidence, attendance, readiness, migration, and final-readiness skills	HR/Payroll specialist; Data Privacy Officer; Backend Architect; Financial Controller	HRIS-owned boundaries; compatibility migration; fine-grained permissions; certified inputs; payment/declaration/accounting-close proof
7. Country-Pack Production Readiness	Convert fail-closed statutory foundations into expert-reviewed, versioned country support.	006 country-pack-factory; 015 country-adapter-pilot; payroll-country-pack-engine; statutory-service-modernizer	Legal Compliance Checker; Tax Strategist; Payroll specialist; external statutory expert	Hashed sources; expert approval; effective dates/versioning; certified sandbox adapter; unsupported automation blocked

6. Stage D — Production Assurance and Rollout

8. Production Operations and Controlled Rollout	Prove secrets, CI, deployment, scheduler, alerts, backup/restore, DR, monitoring, pilots, and rollback.	017 enterprise-release-gate; narrow-enforce-external-readiness; certification-assurance-hardener; provider-health-recon-ops	SRE; Senior SecOps Engineer; Incident Response Commander; Compliance Auditor; Reality Checker	Clean-commit CI; production secrets; alert acknowledgement; restore/DR exercise; controlled pilot; rollback proof; signed release decision

7. Agent Coordination Contract

- The primary Program Orchestrator owns sequencing, phase status, scope, and handoffs.
- Explorers remain read-only and return evidence with file references, confidence, and contradictions.
- Only one implementation agent owns writable files in a slice.
- Security and financial reviewers cannot certify their own implementation.
- The Reality Checker defaults to NO_GO until required proof is present.
- External experts approve statutory meaning; agents may record evidence but cannot self-certify legal compliance.
- Subagents inherit current phase scope and may not expand into later phases.
- Any shared-schema or shared-control-plane overlap forces sequential execution.

8. Program Interfaces and Artifacts

```
PhaseStatus =  
NOT_STARTED | DISCOVERY | PLANNED | APPROVED |  
IN_PROGRESS | BLOCKED | VERIFIED | RELEASED  
  
GateDecision =  
GO | GO_WITH_GATES | NO_GO  
  
EvidenceClass =  
PROVEN | IMPLEMENTED_NOT_PROVEN | PARTIAL |  
DOCUMENTED_ONLY | EXTERNAL_DEPENDENCY | CONTRADICTED_OR_STALE
```

Each phase handoff packet records

- Phase and slice identifier; source commit SHA; dirty-worktree state.
- Objective, scope, non-goals, owner, reviewers, selected skills, and agent roles.
- Changed boundaries/files; migrations/backfills; rollback.
- Commands, exit codes, environment, timestamps, hashes, and artifact paths.
- Passed, failed, skipped, blocked, and external checks.
- Residual risks, human decisions, gate decision, and next authorized skill.

```
what-next/enterprise-transformation/  
  program-status.md  
  program-status.json  
  phases/  
    phase-00/<run-id>/  
    phase-01/<run-id>/  
    ...  
    phase-08/<run-id>/
```

Every completed phase produces

- Markdown execution report and matching PDF.
- JSON gate/evidence manifest.
- Focused test and smoke outputs.

- Redacted screenshots where UI is involved.
- Migration and rollback evidence where schema is involved.

9. Refined Master Orchestrator Prompt

Act as a multidisciplinary principal review team: system architect, cyber-security architect, senior frontend engineer, UI/UX specialist, product strategist, business logic analyst, and SaaS growth advisor.

Workspace:

E:\ohada saas\Focused projects\stoquify

Governing assessment:

docs/stoquify ambitions/

STOQUIFY_ENTERPRISE_PROOF_DRIVEN_ARCHITECTURE_ASSESSMENT_2026-07-24.md

Mission:

Transform Stoquify through the approved nine-phase proof-driven enterprise program. Select and execute only the next authorized bounded slice. Preserve one authoritative owner for every business fact and require every material workflow to pass through authorization, validation, an idempotent transaction, domain truth, business-event/outbox evidence, provenance/hash, assurance, exception ownership, trusted read models, and invalidation when evidence changes.

Operating rules:

1. Read AGENTS.md, Git status, the program register, the governing assessment, current graph artifacts, recent readiness reports, and the selected skill.
2. Treat existing changes as user-owned and never overwrite unrelated work.
3. Classify evidence as PROVEN, IMPLEMENTED_NOT_PROVEN, PARTIAL, DOCUMENTED_ONLY, EXTERNAL_DEPENDENCY, or CONTRADICTED_OR_STALE.
4. Use the authoritative skill assigned to the current phase.
5. Run parallel agents only for read-only, non-overlapping discovery or independent verification.
6. Assign all writes in a slice to one implementation owner.
7. Stop before implementation until the phase plan has a recorded GO or GO_WITH_GATES decision.
8. Implement the smallest high-leverage slice. Do not refactor unrelated code, weaken controls, invent statutory rules, or advance into another phase.
9. Verify with focused tests, domain gates, cross-domain assurance, diff review, and independent verification.
10. Save Markdown, PDF, and JSON evidence under the phase run directory.
11. Update the program register and stop for explicit phase approval.
12. Never describe repository evidence as provider, statutory, hardware, or production certification without matching external proof.

For the current run, produce:

- current phase and prerequisite decision;
- evidence and contradiction register;
- bounded slice plan;
- assigned skills and agents;
- implementation ownership;
- migration/backfill and rollback plan;
- verification matrix;
- required artifacts;
- GO, GO_WITH_GATES, or NO_GO decision;
- exact next authorized action.

Do not execute more than one implementation slice or cross a phase gate in one run.

10. Test and Acceptance Plan

Program-level acceptance

- Every phase has prerequisites, one lead skill, specialist agents, success criteria, rollback, and a blocking exit gate.
- No phase begins while an earlier required phase is BLOCKED.
- No implementation agent certifies its own work.
- No concurrent writers touch shared schema, control-plane, authorization, or accounting boundaries.
- Every readiness claim is tied to commit, environment, commands, and evidence hashes.
- Static gates remain explicitly distinct from runtime and external proof.
- A failed or missing critical check produces NO_GO, not a warning-only release.
- /compact occurs only after the handoff packet and program register are durable.

Phase-level verification

- Focused unit and integration tests.
- Tenant isolation, RBAC, entitlement, fresh-auth, maker-checker, and redaction tests.
- Idempotency, replay, concurrency, reversal, and invalidation tests.
- PostgreSQL migration and persistence smoke where applicable.
- Desktop/mobile browser evidence for operator workflows.
- Relevant domain gates, then typecheck, focused lint, and justified policy gates.
- Independent evidence review before release.

11. Assumptions and Defaults

- The deliverable is the full execution package: master prompt, phase briefs, skill/agent matrix, gates, and handoffs.
- Progression uses explicit human approval after every phase.
- Delivery uses a sequential implementation spine with parallel read-only discovery and verification.
- Phase 1 retains the read-only Analytics entitlement pilot.
- aqstoqflow-hris-payroll-00-orchestrator is the canonical HRIS/payroll router; the older hrpayroll suite is compatibility-only.
- The legacy module-commercialization router delegates to aqstoqflow-module-control-plane-orchestrator.
- Schedule is evidence-driven, not calendar-driven.
- No statutory, provider, production, or external readiness is inferred from repository tests.

12. Program Decision

GO_WITH_GATES	The staged proof-driven enterprise transformation program.
NO_GO	Unrestricted production, statutory certification, or agent activation until Phase 8 passes.

The next authorized move is Phase 0: establish a reproducible evidence baseline without losing or overwriting user-owned changes. No later phase may begin until its prerequisite gate is explicitly approved.